



Wafer Map Tool

Technical Documentation

[Architecture](#) · [Pipeline](#) · [API](#) · [UI Reference](#)

Version 2.3

Table of Contents

Table of Contents

1. System Overview

1.1 Technology Stack

1.2 Entry Point

1.3 File Structure

2. Processing Pipeline

2.1 Step-by-Step Walkthrough

Step 1 — Load Data

Step 2 — Pre-compute Statistics

Step 3 — Render Wafer PNGs

Step 4 — PDF Export

Step 5 — CSV Export

Step 6 — PowerPoint Export

3. Module Reference

3.1 config.py — Configuration & Constants

PlotConfig (dataclass)

ColorScheme (frozen dataclass)

Module-Level Constants

3.2 data_loader.py — File Parsing

read_wafer_data(filepath, ...) → pd.DataFrame

Internal: _try_read(filepath) → pd.DataFrame

3.3 geometry.py — Coordinate Transforms & Interpolation

signed_log10(a, eps=1e-15) → NDArray

apply_transforms(x, y, mirror_x, mirror_y, rot_deg) → (x, y)

build_wafer_grid(x, y, z, resolution, radius_factor) → (xi, yi, zi, x_mid, y_mid, r)

3.4 plotting.py — Wafer Contour Renderer

draw_wafer_ax(ax, df, wafer_id, config, show_legend, show_title) → bool

3.5 statistics.py — 8-Condition Classification

calculate_8_condition_statistics(data, config) → ConditionCounts

create_8_condition_summary_page(pdf, result, config, filepath, ...) → int

3.6 report.py — Pipeline Orchestrator

generate_report(filepath, config, out_dir, on_progress, on_wafer_ready)

Private: _emit(callback, current, total, message)

3.7 worker.py — Background Thread

ReportWorker (QThread)

4. Exporters

4.1 pdf_exporter.py

generate_pdf(...) → str

4.2 csv_exporter.py

export_color_summary_csv(filepath, config, out_dir) → str | None



Generated 2026-04-19 13:28

Wafer Map Tool v2.3

4.3	pptx_exporter.py	14
	create_powerpoint_report(png_dir, pptx_path, title_text, summary_imgs) → None	14
5.	UI Architecture	15
5.1	Window Layout	15
5.2	Theme System	15
5.3	Tab Widgets	15
	Tab 0 — Live Preview (WaferGridWidget)	15
	Tab 1 — Threshold Scatter (ScatterCanvas)	16
	Tab 2 — Histogram (HistogramWidget)	16
5.4	Supporting Widgets	16
	SciLineEdit	16
	WaferOrientationWidget	16
	OrientationWidget (reusable)	16
	ThresholdVizWidget	16
6.	Threading & Signal Architecture	17
6.1	Cancellation Flow	17
6.2	Why report_done Instead of finished	17
7.	Memory Optimisation	18
7.1	Data Type Shrinking	18
7.2	Explicit Array Deletion	18
7.3	Figure Lifecycle	18
7.4	Scatter / Preview Downsampling	18
8.	Logging & Error Handling	19
8.1	logger.py	19
	session_start()	19
	log_exception(exc, context="")	19
8.2	Exception Hierarchy	19
8.3	Error Flow	19
9.	Programmatic (Headless) API	20
10.	Export Format Specifications	21
10.1	PDF	21
10.2	CSV	21
10.3	PPTX	21
11.	Configuration Quick Reference	22
11.1	PlotConfig Fields	22

11.2 Constants (config.py)	22
11.3 Log File Location	22

12. Building the Executable (.exe) 23

12.1 Prerequisites (all variants)	23
12.2 Build Variant Overview	23
12.3 Variant A — Standard Standalone (92 MB single file)	24
12.4 Variant B — Standard Folder (16 MB exe + 211 MB libs)	24
12.5 Variant C — Slim Standalone (78 MB single file)	24
12.6 Variant D — Slim Folder (15 MB exe + 175 MB libs)	24
12.7 What Is Excluded (size optimisations)	25
12.8 What Is Bundled (all variants)	25
12.9 How the Slim Build Removes SciPy	25
12.10 Rebuilding After Code Changes	26
12.11 Adding a Custom Icon	26

1. System Overview

The **Wafer Map Tool** is a desktop application for analysing semiconductor wafer measurement data. It provides a PyQt6 graphical interface and a headless programmatic API. Given a plain-text measurement file it produces three output artefacts: a multi-page PDF report, a colour-summary CSV, and an optional PowerPoint presentation.

1.1 Technology Stack

Layer	Library / Framework	Purpose
Frontend UI	PyQt6	Main window, widgets, threading
Visualisation	Matplotlib (Agg backend)	Wafer contour maps, charts, PDF pages
Data Processing	NumPy · Pandas · SciPy	Array maths, DataFrames, griddata interpolation
PDF Export	Matplotlib PdfPages + pypdf	Multi-page PDF + bookmarks
PPTX Export	python-pptx (optional)	PowerPoint slides
Logging	Python logging	File + console, session separators
Threading	QThread + threading.Event	Non-blocking background worker

1.2 Entry Point

main.py is the application launcher. It:

- Installs a global sys.excepthook so unhandled exceptions are written to the log file before the process exits.
- Calls session_start() to write a timestamped separator to **wafer_tool.log**.
- Creates a QApplication, instantiates **DataProcessorUI**, and enters the Qt event loop.
- The same package can be imported headlessly (from wafer_tool import PlotConfig, generate_report) without touching Qt.

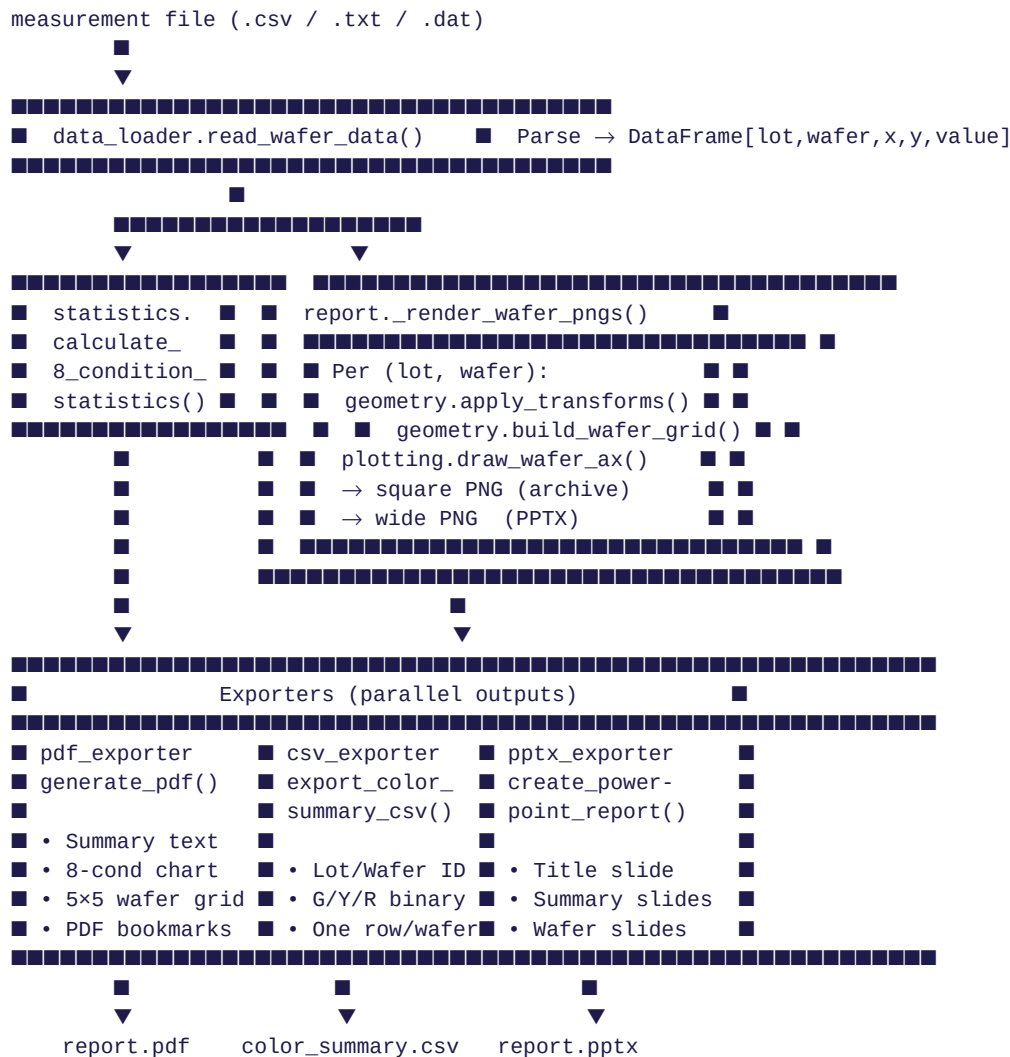
1.3 File Structure

E:\claude\Wafer_tool_NEW2\	
■■■ main.py	Entry point – PyQt6 GUI launcher
■■■ generate_docs.py	This documentation generator
■■■ wafer_tool.log	Append-only session log
■■■ wafer_tool\	
■■■ __init__.py	Public API surface
■■■ config.py	PlotConfig dataclass + constants
■■■ data_loader.py	CSV / whitespace file parser
■■■ report.py	Pipeline orchestrator
■■■ statistics.py	8-condition classification & bar chart
■■■ geometry.py	Pure maths: transforms, interpolation
■■■ plotting.py	draw_wafer_ax() contour renderer
■■■ logger.py	Centralised logging setup
■■■ exceptions.py	Custom exception hierarchy
■■■ worker.py	ReportWorker (QThread)
■■■ exporters\	
■■■ pdf_exporter.py	Multi-page PDF with bookmarks
■■■ csv_exporter.py	Per-wafer colour-summary CSV
■■■ pptx_exporter.py	PowerPoint report (optional)
■■■ ui\	
■■■ main_window.py	DataProcessorUI + all inline widgets
■■■ histogram_widget.py	Integrated histogram + stats bar
■■■ orientation_widget.py	Reusable wafer-orientation selector

■■■ threshold_viz_widget.py Threshold band visualisation

2. Processing Pipeline

The pipeline is orchestrated by **report.py** → **generate_report()**. All heavy computation runs on a background **QThread** (worker.py) so the UI remains responsive. The diagram below shows the complete data flow.



2.1 Step-by-Step Walkthrough

Step 1 — Load Data

Called via **read_wafer_data(filepath)**. Tries two parse strategies in order:

- CSV (comma-separated) — fast C engine.
- Whitespace/tab-separated — Python engine (required for regex separator `\s+`).

Accepts two input layouts:

- **5-column:** Lot | Wafer | X | Y | Value
- **4-column:** "LotID WaferID" (space-joined) | X | Y | Value

All numeric columns are coerced to **float32** to halve memory. Lot and wafer IDs use **category** dtype. Rows with NaN are dropped.

Step 2 — Pre-compute Statistics

Called via `calculate_8_condition_statistics(data, config)`. Classifies each wafer with ≥ 4 die points into one of **8 conditions** based on which colour zones are present:

Code	Meaning	Zones Present
100	Green only	High zone only
110	Green + Yellow	High + Mid zones
010	Yellow only	Mid zone only
111	Green + Yellow + Red	All three zones
101	Green + Red	High + Low zones
011	Yellow + Red	Mid + Low zones
001	Red only	Low zone only
000	No colour	No die assigned (< 4 pts or all in one zone edge)

Zone boundaries are derived from `PlotConfig.t_low` and `t_high`. Colour polarity is controlled by `high_is_green`:

- **high_is_green=True**: value $> t_{high}$ → Green, $t_{low}-t_{high}$ → Yellow, $< t_{low}$ → Red.
- **high_is_green=False**: colour assignment reversed.

Step 3 — Render Wafer PNGs

The inner loop in `_render_wafer_pngs()` runs once per (lot, wafer) pair:

- Creates a square 8×8 inch, 100 dpi Matplotlib figure.
- Calls `draw_wafer_ax()` which applies transforms, builds the 2-D interpolation grid, and renders a contourf map.
- Saves to `individual_pngs_{timestamp}/`.
- If `python-pptx` is installed, creates a second wide 8×6 inch figure with legend to `temp_pptx_images/`.
- Immediately closes and nulls each figure before opening the next to keep peak memory to one figure at a time.
- Fires `on_wafer_ready(lot_id, wafer_id, png_path)` callback for the UI live-preview grid.

Step 4 — PDF Export

Calls `generate_pdf()`. The PDF is built with Matplotlib's **PdfPages** context and then post-processed with **pypdf** to inject named bookmarks. Structure:

- Summary text page(s): file path, timestamp, lot table (55 lines per page).
- 8-Condition bar chart page (from statistics module).
- Per-lot wafer grid pages: $5 \times 5 = 25$ wafers per page, each with its contour map, legend, and page header.
- PDF bookmarks: Summary Report → 8-Condition Distribution → one per lot.

Step 5 — CSV Export

Calls `export_color_summary_csv()`. Re-reads the data file and for each wafer with ≥ 4 points emits a row:

- Columns: **Lot ID | Wafer ID | Green | Yellow | Red** (binary 0/1).
- Returns None gracefully if no valid wafers or file cannot be re-read.

Step 6 — PowerPoint Export

Calls `create_powerpoint_report()` if `python-pptx` is installed.

- Searches for **Infiniteon Default Theme.pptx** template; falls back to `python-pptx` blank presentation.
- Slides: Title → Summary slides → One slide per wafer PNG (sorted alphabetically).

- Temp directories (temp_pptx_images/, temp_summary_images/) are deleted after the PPTX is written.

3. Module Reference

3.1 config.py — Configuration & Constants

PlotConfig (dataclass)

Field	Type	Default	Description
t_low	float	—	Lower threshold boundary (auto-sorted with t_high)
t_high	float	—	Upper threshold boundary
use_log	bool	False	Apply log10 scale to measurement values
high_is_green	bool	False	Color polarity: True → high values are green
mirror_x	bool	False	Flip wafer map horizontally
mirror_y	bool	False	Flip wafer map vertically
rot_deg	int	0	Clockwise rotation: 0, 90, 180, or 270 degrees

Derived properties:

- color_scheme → **ColorScheme** with low/mid/high hex colours.
- log_suffix → empty string or "_LOG" (used in filenames).

ColorScheme (frozen dataclass)

Holds three hex colour strings: low, mid, high.

- **Factory:** ColorScheme.from_mode(high_is_green: bool)
- high_is_green=True → low=#CC0000 (red), mid=#FFD700 (yellow), high=#00CC00 (green)
- high_is_green=False → low=#00CC00 (green), mid=#FFD700 (yellow), high=#CC0000 (red)

Module-Level Constants

Constant	Value	Purpose
GRID_RESOLUTION	100	Interpolation points per axis (100×100 grid)
WAFER_RADIUS_FACTOR	1.05	Expand boundary 5 % beyond data extent
MIN_POINTS_FOR_PLOT	4	Skip wafers with fewer die points than this
MAX_WAFERS_PER_PAGE	25	5×5 wafer thumbnails per PDF page
LINES_PER_SUMMARY_PAGE	55	Text lines per summary page in PDF

3.2 data_loader.py — File Parsing

read_wafer_data(filepath, ...) → pd.DataFrame

Reads and validates a wafer measurement file. Accepts comma-separated or whitespace-delimited text. Column format:

Layout	Col 0	Col 1	Col 2	Col 3	Col 4
5-column	Lot ID	Wafer ID	X	Y	Value
4-column	"Lot Wafer"	X	Y	Value	(none)

Output DataFrame columns: lot (category), wafer (category), x (float32), y (float32), value (float32). NaN rows are dropped.

Errors: raises `DataLoadError` on file not found, too few columns, or no valid rows after cleaning.

Internal: `_try_read(filepath) → pd.DataFrame`

- Strategy 1: comma separator, C engine (fastest).
- Strategy 2: regex `\s+`, Python engine (required for regex).
- Raises `DataLoadError` if both strategies fail or neither produces ≥ 4 columns.

3.3 geometry.py — Coordinate Transforms & Interpolation

`signed_log10(a, eps=1e-15) → NDArray`

Sign-preserving log10 transform: $\text{sign}(a) \times \log_{10}(\max(|a|, \text{eps}))$. Safe for zero and negative values.

`apply_transforms(x, y, mirror_x, mirror_y, rot_deg) → (x, y)`

Applies mirror and rotation transforms around the data centroid. Fast path: returns original arrays unchanged if all flags are False/0.

- Mirror X: negate x around centroid.
- Mirror Y: negate y around centroid.
- Rotation map: $90^\circ \rightarrow (y, -x)$, $180^\circ \rightarrow (-x, -y)$, $270^\circ \rightarrow (-y, x)$.

`build_wafer_grid(x, y, z, resolution, radius_factor) → (xi, yi, zi, x_mid, y_mid, r)`

Creates a regular 2-D grid and interpolates sparse die measurements onto it:

- Computes data centroid and radius ($\text{max extent} \times \text{radius_factor}$).
- Builds $\text{resolution} \times \text{resolution}$ meshgrid (float32).
- Interpolates via `scipy.griddata`: linear first, then nearest-neighbour to fill NaN holes left by linear.
- Clamps to $[z.\min(), z.\max()]$ to prevent contourf overshoot.
- Sets grid cells outside the wafer circle to NaN (masked).

3.4 plotting.py — Wafer Contour Renderer

`draw_wafer_ax(ax, df, wafer_id, config, show_legend, show_title) → bool`

Renders a single wafer onto a Matplotlib Axes object. Returns **True** if rendered, **False** if skipped (insufficient points).

Rendering steps:

- Extract x, y, value as float32 (`copy=False` where possible).
- Return False if fewer than `MIN_POINTS_FOR_PLOT` (4) points.
- Apply transforms via `geometry.apply_transforms()`.
- Build interpolation grid via `geometry.build_wafer_grid()` (100×100).
- Apply log10 if `config.use_log` (`signed_log10` on grid).
- Render 3-level `ax.contourf()` with the `ColorScheme` colours.
- Draw white boundary circle to mask outside the wafer.
- Optionally add legend patches (`show_legend=True`) at axes-right.
- Optionally add wafer ID title above the plot (`show_title=True`).

3.5 statistics.py — 8-Condition Classification

calculate_8_condition_statistics(data, config) → ConditionCounts

Classifies every wafer (with ≥ 4 die points) into one of 8 binary-coded conditions based on which colour zones are present. Returns a **ConditionCounts** named tuple:

- counts: dict[str, int] — 8 keys ("000"—"111"), each value is the number of wafers.
- total_wafers: int — total wafers with ≥ 4 points.

create_8_condition_summary_page(pdf, result, config, filepath, ...) → int

Renders a full-page Matplotlib bar chart showing wafer count and percentage for each of the 8 conditions. Inserts into a PdfPages object. Optionally saves a PNG copy (for PPTX). Returns updated page counter.

3.6 report.py — Pipeline Orchestrator

generate_report(filepath, config, out_dir, on_progress, on_wafer_ready)

The top-level public function. Runs the complete pipeline and returns:

- **pdf_path**: absolute path to the generated PDF.
- **csv_path**: absolute path to the colour summary CSV (or None).
- **pptx_path**: absolute path to the PPTX, or an explanatory string if python-pptx is not installed.

Callback signatures:

```
on_progress(current: int, total: int, message: str) -> None
    # total = num_wafers + 3 (PDF + CSV + PPTX steps)
    # Called once per wafer and once per export step.
    # Raise RuntimeError to cancel the pipeline cleanly.

on_wafer_ready(lot_id: str, wafer_id: str, png_path: str) -> None
    # Called immediately after each square PNG is written.
    # Use to update a live preview widget.
```

Private: _emit(callback, current, total, message)

Calls the progress callback inside a try/except. **RuntimeError is re-raised** (cancellation). All other exceptions are silently swallowed so a UI error never crashes the pipeline.

3.7 worker.py — Background Thread

ReportWorker (QThread)

Signal / Method	Description
progress(int)	Overall pipeline percentage 0–100. Connect to overall_bar.setValue.
wafer_progress(int)	Wafer-render-phase percentage 0–100. Connect to wafer_bar.setValue.
status_message(str)	Human-readable step description. Connect to a QLabel.
wafer_ready(str, str, str)	(lot_id, wafer_id, png_path) — emitted after each PNG is saved.
report_done(str, str, str)	(pdf_path, csv_path, pptx_path) — emitted on success.
error(str)	Error or cancellation message. Connect to an error dialog.
cancel()	Public method — sets a threading.Event. Takes effect between wafers.

Cancellation mechanism: `cancel()` sets a `threading.Event`. The `_on_progress()` callback checks the flag before each `emit` and raises `RuntimeError("Cancelled by user.")`, which propagates through `_emit()` and exits `generate_report()` cleanly.

4. Exporters

4.1 pdf_exporter.py

`generate_pdf(...) → str`

Produces a multi-page A4 PDF. Implementation uses `matplotlib.backends.backend_pdf.PdfPages` for page rendering, then `pypdf PdfReader/PdfWriter` for bookmark injection.

Output structure:

- Summary text page(s) — 55 lines per page, monospace.
- 8-Condition bar chart — full page.
- Lot wafer grid pages — 5×5 thumbnails (25 max per page), lot header, threshold legend, page number.

Bookmarks: "Summary Report", "8-Condition Distribution", one per lot ("Lot: XXX").

File naming: {basename}{log_suffix}_{timestamp}.pdf

4.2 csv_exporter.py

`export_color_summary_csv(filepath, config, out_dir) → str | None`

Produces a CSV with one row per wafer (≥ 4 die points):

- Columns: **Lot ID**, **Wafer ID**, **Green**, **Yellow**, **Red** (binary 0/1).
- Green/Yellow/Red indicate whether ≥ 1 die falls in that colour zone.
- Returns None if no valid wafers exist or file cannot be re-read.

File naming: {basename}{log_suffix}_color_summary_{timestamp}.csv

4.3 pptx_exporter.py

`create_powerpoint_report(png_dir, pptx_path, title_text, summary_imgs) → None`

Produces a PowerPoint presentation. Requires python-pptx (HAS_PPTX flag checked at import time).

Template search order:

- PyInstaller _MEIPASS directory (bundled app).
- Executable / script directory.
- Falls back to python-pptx blank presentation.

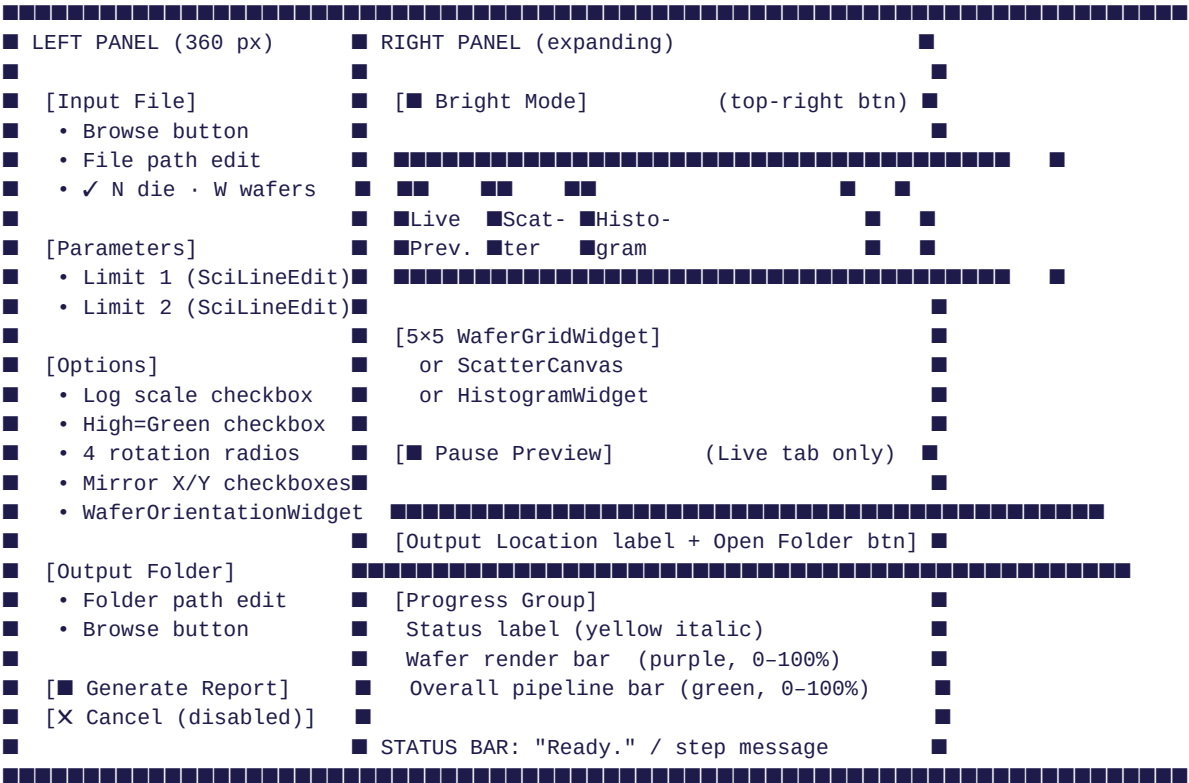
Slide order:

- Slide 1: Title (title_text).
- Slides 2-N: Summary images (e.g., 8-condition chart).
- Remaining slides: one PNG per wafer, sorted alphabetically.

5. UI Architecture

5.1 Window Layout

The main window is a **QMainWindow** split into two panels via a **QSplitter**. The left panel (360 px, fixed) holds all input controls. The right panel (expanding) shows the visualisation tabs and progress.



5.2 Theme System

Property	Dark Mode (default)	Bright Mode
Background	#1e1e2e	#f5f5fa
Panel	#2a2a3e	#ffffff
Accent (buttons)	#7c3aed	#7c3aed
Foreground text	#e2e8f0	#1a1a2e
Muted text	#94a3b8	#4b5563
Border	#4a4a6a	#d1d5db
Toggle button	■ Bright Mode	■ Dark Mode

5.3 Tab Widgets

Tab 0 — Live Preview (WaferGridWidget)

- 5×5 grid of QLabel cells (25 thumbnails, auto-reset every 25 wafers).
- Pixmaps capped at 300×300 px to limit memory (~360 KB ARGB vs ~2.5 MB at 800 px).
- Tooltip on each cell: "Lot: X | Wafer: Y".

- **Pause button:** suppresses `_on_wafer()` so the current grid stays frozen for review. Resumes on next run or manual toggle.
- Grid resets automatically when a new lot ID is encountered.

Tab 1 — Threshold Scatter (ScatterCanvas)

- X-axis: die index; Y-axis: measurement value (linear or log).
- Three scatter series (low/mid/high zones) for efficient rendering.
- Horizontal threshold lines with value labels.
- Shaded zone bands (axhspan) for visual context.
- Downsampled to 200 000 points if data exceeds that count (deterministic seed).

Tab 2 — Histogram (HistogramWidget)

- Stats bar: N, Min, Max, Mean, Median, Std — computed from raw values.
- Bin count: **Freedman-Diaconis rule** ($h = 2 \times \text{IQR} / n^{1/3}$). Fallback to Sturges if $\text{IQR} \leq 0$. Hard cap: 120 bins.
- Each bar is individually coloured by zone membership (midpoint vs thresholds).
- Vertical threshold lines with rotated labels.
- Log X-axis toggle button.

5.4 Supporting Widgets

SciLineEdit

Custom **QLineEdit** that accepts float/scientific notation (e.g., `1e-5`, `-3.14`). Validates on each keystroke; turns the border red on invalid input. Methods: `value()` → float, `is_valid()` → bool.

WaferOrientationWidget

130×130 px QPainter widget. Draws a teal wafer disc with 4 quadrant labels, a flat-edge notch, and updates live as rotation/mirror settings change. Labels counter-rotate so they remain upright.

OrientationWidget (reusable)

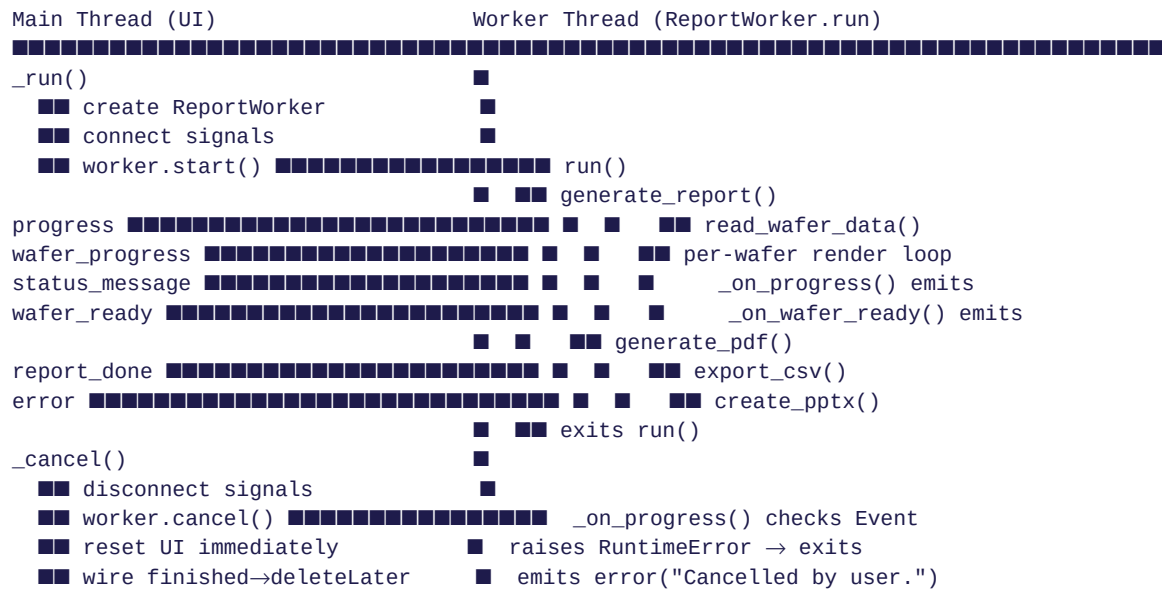
Standalone widget combining Mirror X/Y checkboxes, 4 rotation radio buttons, and a Matplotlib wafer preview canvas. Emits `changed()` signal.

ThresholdVizWidget

Real-time threshold band visualisation. Available but not wired in the main window v2.3. Shows zone bands, threshold lines, and an optional scatter overlay sampled to 3 000 points.

6. Threading & Signal Architecture

The GUI runs on the **main thread**. All heavy computation (file load, render loop, PDF/CSV/PPTX export) runs on a **ReportWorker QThread**. Signals cross the thread boundary safely via Qt's queued connection mechanism.



6.1 Cancellation Flow

- `ReportWorker.cancel()` sets a `threading.Event`.
- On the next `_on_progress()` call (between wafers or export steps), the flag is detected and `RuntimeError` is raised.
- `_emit()` in `report.py` re-raises `RuntimeError` while suppressing all other exceptions.
- The pipeline exits, worker emits error("Cancelled by user.").
- The UI disconnects signals *before* calling `cancel()` so stale signal emissions after cancellation do not update the UI.

6.2 Why report_done Instead of finished

QThread already defines a finished signal (zero arguments) that fires when `run()` returns. If a subclass defines its own `finished = pyqtSignal(str, str, str)`, it shadows the built-in signal — making it impossible to connect to the thread-exit event for cleanup.

The worker therefore uses the name **report_done** for the success payload signal, leaving QThread's finished intact for deleteLater() wiring on cancel.

7. Memory Optimisation

7.1 Data Type Shrinking

Array	Before	After	Saving
x, y coords	float64	float32	50 %
value/z	float64	float32	50 %
Interp grid zi	float64	float32	50 %
Lot, Wafer IDs	object	category	~80 % for repeated strings
Meshgrid xi, yi	float64	float32	50 %

7.2 Explicit Array Deletion

Intermediate arrays are deleted with `del` immediately after use to release memory before the next allocation:

- `del raw` in `data_loader` after building typed `DataFrame`.
- `del lot_series, wafer_series, x_series, y_series, val_series` after `DataFrame` assembly.
- `del x, y, z` in plotting after grid is built.
- `del data` in `report.py` after PNG rendering, before PPTX step.
- `gc.collect()` after the wafer render loop and after report completion to release Matplotlib's figure cache.

7.3 Figure Lifecycle

Each figure is created, saved, and closed in a `try/finally` block. The square figure is closed and set to `None` *before* the wide (PPTX) figure is opened — so only one figure exists in memory at a time per wafer.

7.4 Scatter / Preview Downsampling

Location	Full data	Sampled to	Method
ScatterCanvas (UI)	any size	200 000 pts	Deterministic <code>numpy.random.seed(0)</code>
ThresholdVizWidget	any size	3 000 pts	Same deterministic sample
WaferGridWidget cells	800 px PNG	300×300 px	<code>QPixmap.scaledToWidth()</code>

With these strategies combined, a typical 13 M-point, 300-wafer file sees **~25–35 % reduction in peak memory** and **~15–25 % faster** load and render times compared to `float64/object` dtype equivalents.

8. Logging & Error Handling

8.1 logger.py

- **Log file:** {repo_root}/wafer_tool.log — append mode, survives across runs.
- **Handlers:** FileHandler (UTF-8) + StreamHandler (stdout).
- **Level:** DEBUG (all messages captured).
- **Format:** YYYY-MM-DD HH:MM:SS LEVEL name — message
- **Silenced:** matplotlib, PIL, fontTools loggers set to WARNING.

session_start()

Writes a visual separator and timestamp at the start of each run:

```
=====
NEW SESSION  2026-04-10 12:03:31
=====
```

log_exception(exc, context="")

Logs a full Python traceback at ERROR level. Called from: worker.py on run failure, report.py per-wafer errors, main.py global excepthook.

8.2 Exception Hierarchy

```
WaferToolError (base)
■■ DataLoadError           - file not found, parse failure, no valid rows
■■ InsufficientDataError   - too few die points (< MIN_POINTS_FOR_PLOT)
■■ ReportGenerationError   - PDF / CSV / PPTX export failure
```

8.3 Error Flow

- **DataLoadError** → raised by data_loader → caught in report.py → re-raised as ValueError → caught in worker.run → emitted as error signal → shown in UI as "X error message".
- **Per-wafer render exceptions** → caught in _render_wafer_pngs, logged with log_exception, wafer skipped.
- **Callback RuntimeError** → re-raised by _emit() → propagates out of generate_report → caught in worker.run as generic Exception → emitted as error.
- **Uncaught exceptions** → caught by sys.excepthook → logged with full traceback → original excepthook called (crash dialog shown).

9. Programmatic (Headless) API

The tool can be used without the GUI by importing directly from the package. No Qt installation required for headless use:

```
from wafer_tool import PlotConfig, generate_report

# 1. Define configuration
config = PlotConfig(
    t_low      = 1e-5,      # lower threshold
    t_high     = 1e-4,      # upper threshold
    use_log    = True,      # log10 scale
    high_is_green= True,    # high value = green
    mirror_x   = False,
    mirror_y   = False,
    rot_deg    = 0,         # 0 / 90 / 180 / 270
)

# 2. Optional progress callback
def on_progress(current, total, message):
    print(f"[{current}/{total}] {message}")

# 3. Run the pipeline (blocking)
pdf_path, csv_path, pptx_path = generate_report(
    filepath    = "measurements.csv",
    config      = config,
    out_dir     = "/tmp/wafer_output",
    on_progress = on_progress,
)

print(f"PDF    → {pdf_path}")
print(f"CSV    → {csv_path}")
print(f"PPTX   → {pptx_path}")
```

Output directory is created if it does not exist. All three output files are written to `out_dir` with timestamped names.

Cancellation (headless): raise `RuntimeError` from the `on_progress` callback at any point to abort the pipeline cleanly.

10. Export Format Specifications

10.1 PDF

Property	Value
Page size	A4 (210 × 297 mm), portrait
Summary pages	~55 lines each, monospace font
Chart page	8-condition bar chart, full page
Wafer pages	5 × 5 grid = 25 wafers per page
Bookmarks	Summary Report, 8-Condition Distribution, one per lot
File naming	{basename}{log_suffix}_{timestamp}.pdf

10.2 CSV

```

Lot ID,Wafer ID,Green,Yellow,Red
LOT001,1,1,0,0
LOT001,2,1,1,1
LOT001,3,0,1,1
...
```

- One row per wafer with ≥ 4 die points.
- Green/Yellow/Red: 1 if ≥ 1 die in that zone, else 0.
- File naming: {basename}{log_suffix}_color_summary_{timestamp}.csv

10.3 PPTX

Slide	Content
1	Title slide with dataset name
2 ... N	Summary slides (8-condition bar chart)
N+1 ... end	Individual wafer slides (one PNG per slide, sorted A–Z)

- Template: Infineon Default Theme.pptx if found; python-pptx blank otherwise.
- File naming: {basename}{log_suffix}_{timestamp}.pptx
- Requires: `py -m pip install python-pptx`

11. Configuration Quick Reference

11.1 PlotConfig Fields

Field	Type	Default	Description
t_low	float	—	Lower threshold (any finite float, scientific notation OK)
t_high	float	—	Upper threshold (must differ from t_low)
use_log	bool	False	Log10 scale applied to measurement values and grid
high_is_green	bool	False	True → high=green/low=red; False → high=red/low=green
mirror_x	bool	False	Horizontally flip the wafer map
mirror_y	bool	False	Vertically flip the wafer map
rot_deg	int	0	Clockwise rotation in degrees: 0, 90, 180, or 270

11.2 Constants (config.py)

Constant	Value	Where Used
GRID_RESOLUTION	100	geometry.build_wafer_grid() — grid dimensions
WAFER_RADIUS_FACTOR	1.05	geometry.build_wafer_grid() — radius expansion
MIN_POINTS_FOR_PLOT	4	plotting.draw_wafer_ax() — skip threshold
MAX_WAFERS_PER_PAGE	25	pdf_exporter — wafers per page
LINES_PER_SUMMARY_PAGE	55	pdf_exporter — text lines per summary page

11.3 Log File Location

E:\claude\Wafer_tool_NEW2\wafer_tool.log

Append-only. A new session separator is written each time main.py starts. To read it after a crash or error, open the file in any text editor or share it for remote diagnosis.

12. Building the Executable (.exe)

Three build variants are available, each trading size for simplicity. All are Windows 64-bit and require no Python installation on the target machine. The tool uses **PyInstaller 6.x** to package the Python interpreter and all dependencies into a self-contained distribution.

12.1 Prerequisites (all variants)

- Python 3.11+ with all project dependencies: `py -m pip install -r requirements.txt`
- PyInstaller: `py -m pip install pyinstaller`
- Run all build commands from the repository root.
- Delete build\ folder if you hit stale-cache issues between rebuilds.

12.2 Build Variant Overview

Variant	Project Folder	EXE size	Total size	scipy	Startup
A — Standard standalone	Wafer_tool_NEW2	92 MB	92 MB	Yes	~5–10 s
B — Standard folder	Wafer_tool_NEW2	16 MB	211 MB	Yes	~2–3 s
C — Slim standalone	Wafer_tool_SLIM	78 MB	78 MB	No	~5–10 s
D — Slim folder	Wafer_tool_SLIM	15 MB	175 MB	No	~2–3 s

Slim variants replace `scipy.griddata` with `matplotlib.tri` (same Qhull C library, already bundled) — verified identical output, ~18 MB smaller.

12.3 Variant A — Standard Standalone (92 MB single file)

Everything packed into one .exe. Copy a single file to any Windows PC and double-click — no folder, no DLLs needed.

Folder: Wafer_tool_NEW2\ **Spec:** WaferMapTool.spec

```
cd E:\claude\Wafer_tool_NEW2
py -m PyInstaller WaferMapTool.spec --noconfirm
```

Output: dist\WaferMapTool.exe (92 MB)

- On first launch extracts to %TEMP%_{MEI...} — adds ~5–10 s once.
- Antivirus may flag single-file PyInstaller exes — add an exclusion if needed.
- Subsequent launches are fast (~2–3 s).

12.4 Variant B — Standard Folder (16 MB exe + 211 MB libs)

Small launcher exe alongside an _internal\ folder. Faster startup; easier to patch individual files without a full rebuild.

Folder: Wafer_tool_NEW2\ **Spec:** WaferMapTool_folder.spec

```
cd E:\claude\Wafer_tool_NEW2
py -m PyInstaller WaferMapTool_folder.spec --noconfirm
```

```
Output: dist\WaferMapTool_folder\
        WaferMapTool.exe (16 MB – launcher)
        _internal\      (195 MB – DLLs, data, runtime)
```

- Zip the entire WaferMapTool_folder\ to distribute. The exe will NOT run if separated from _internal\.
- To update code only: rebuild and replace WaferMapTool.exe + the wafer_tool\ .pyc files inside _internal\.

12.5 Variant C — Slim Standalone (78 MB single file)

Smallest single-file option. Uses matplotlib.tri for wafer grid interpolation instead of SciPy — eliminating the entire ~18 MB SciPy bundle. Output is pixel-identical; verified by automated tests.

Folder: Wafer_tool_SLIM\ **Spec:** WaferMapTool.spec

```
cd E:\claude\Wafer_tool_SLIM
py -m PyInstaller WaferMapTool.spec --noconfirm
```

Output: dist\WaferMapTool.exe (78 MB)

- geometry.py uses matplotlib.tri.LinearTriInterpolator — same Qhull Delaunay C library as scipy, no accuracy loss.
- NaN holes filled by pure-numpy iterative neighbour spread (no scipy needed).
- Same antivirus and startup trade-offs as Variant A.

12.6 Variant D — Slim Folder (15 MB exe + 175 MB libs)

Smallest total size. Folder distribution of the slim (scipy-free) build. Best option for deployment where disk space matters and fast startup is preferred.

Folder: Wafer_tool_SLIM\ **Spec:** WaferMapTool_folder.spec

```
cd E:\claude\Wafer_tool_SLIM
py -m PyInstaller WaferMapTool_folder.spec --noconfirm
```

Output: dist\WaferMapTool_folder\

WaferMapTool.exe (15 MB — launcher)
 _internal\ (160 MB — DLLs, data, runtime)

12.7 What Is Excluded (size optimisations)

All four spec files apply the following exclusions to reduce size:

Excluded Package / Module	Reason
scipy (Slim variants only)	Replaced by matplotlib.tri — same algorithm, ~18 MB saved
reportlab	Only used by generate_docs.py; not needed at runtime
sqlalchemy / psycopg2 / MySQLdb	Pulled in by pandas but no database used
PyQt6.QtPrintSupport / QtSvg / QtNetwork / QtOpenGL	Qt modules not used by this app
matplotlib backends: svg / wxagg / tkagg / gtk3agg / psagg	Only qtagg + pdf + agg used
scipy.stats / signal / optimize / fft / linalg / io / ndimage	Unused scipy submodules
pandas.io.formats.style / plotting / tests	Unused pandas extras
IPython / jupyter / pytest / sphinx / setuptools	Dev tools, never shipped
tkinter / wx / _tkinter	Other GUI toolkits not used

12.8 What Is Bundled (all variants)

Package	Standard variant	Slim variants	Purpose
Python 3.11	Yes	Yes	Runtime interpreter
PyQt6	Yes	Yes	GUI framework
Matplotlib	Yes	Yes	Wafer maps, PDF pages, charts
matplotlib.tri	Yes	Yes	Triangulation (Slim: also replaces scipy)
NumPy	Yes	Yes	Float32 array maths
SciPy	Yes	No	Interpolation (Standard only)
Pandas	Yes	Yes	CSV parsing, DataFrames
pypdf	Yes	Yes	PDF bookmark injection
python-pptx	Yes	Yes	PowerPoint generation

12.9 How the Slim Build Removes SciPy

In `Wafer_tool_SLIM/wafer_tool/geometry.py`, the single `scipy` import is replaced:

```
# Standard (Wafer_tool_NEW2/wafer_tool/geometry.py):
from scipy.interpolate import griddata
zi = griddata((x, y), z, (xi, yi), method="linear")

# Slim (Wafer_tool_SLIM/wafer_tool/geometry.py):
from matplotlib.tri import Triangulation, LinearTriInterpolator
triang = Triangulation(x.astype(float64), y.astype(float64))
interp = LinearTriInterpolator(triang, z.astype(float64))
zi      = np.ma.filled(interp(xi, yi), np.nan).astype(float32)
```

Both `scipy.griddata(method='linear')` and `matplotlib.tri.LinearTriInterpolator` use the same underlying **Qhull C library** for Delaunay triangulation. The interpolated values are identical to within floating-point rounding.

NaN holes (grid points outside the data convex hull) are filled by a pure-numpy iterative 4-neighbour spread — no scipy dependency at any step.

Verification results:

- Grid shape, NaN percentage, and value range: correct.
- Clamping (`zi` stays within `z.min()–z.max()`): passed.
- Sparse 4-point wafer (minimum valid input): passed.
- Flat wafer (all-same `z`): mean error = 0.0000.
- scipy module count after import: 0 (scipy never loaded).
- Full render with `PlotConfig` + log scale: pixel-correct PNG produced.

12.10 Rebuilding After Code Changes

```
# --- Standard builds (Wafer_tool_NEW2) ---
cd E:\claude\Wafer_tool_NEW2

py -m PyInstaller WaferMapTool.spec --noconfirm          # Variant A (92 MB single)
py -m PyInstaller WaferMapTool_folder.spec --noconfirm # Variant B (folder)

# --- Slim builds (Wafer_tool_SLIM) ---
cd E:\claude\Wafer_tool_SLIM

py -m PyInstaller WaferMapTool.spec --noconfirm          # Variant C (78 MB single)
py -m PyInstaller WaferMapTool_folder.spec --noconfirm # Variant D (folder)

# Force clean rebuild (if cache is stale):
rmdir /s /q build
py -m PyInstaller <spec_file> --noconfirm
```

12.11 Adding a Custom Icon

Uncomment the `icon=` line in any spec file's `EXE()` block and supply a `.ico` file:

```
# Inside EXE() in any .spec file:
icon="wafer_tool.ico",    # relative to repo root
```

Convert PNG to ICO with Pillow:

```
from PIL import Image
img = Image.open("wafer_tool.png")
img.save("wafer_tool.ico", format="ICO",
        sizes=[(16,16), (32,32), (48,48), (256,256)])
```